final_report

May 11, 2026

1 Predicting Urban NO Concentrations from Low-Cost Sensor Arrays

1.0.1 STAT 24620 / FINM 34700 / STAT 32950 — Group Data Project, Spring 2026

Group members: [Name 1], [Name 2], [Name 3]

AI acknowledgment: Claude Code (Anthropic) was used for coding assistance and debugging. All analysis design, interpretation, and written explanations are the authors' own work.

1.1 Research Question

Can a five-sensor electrochemical array (PT08_S1–S5) accurately predict hourly NO concentrations at an Italian urban monitoring site, and does dimensionality reduction via PCA improve prediction relative to directly regressing on the raw sensor readings?

Air-quality monitoring increasingly relies on low-cost sensors alongside certified reference analyzers. If a small number of principal components captures most sensor variance, PC regression may be more stable than full sensor regression — especially when sensors are collinear and subject to environmental drift. We compare OLS, Ridge, Lasso, and PC regression, all augmented with meteorological covariates, on a chronological hold-out test set.

1.2 Dataset

Source: UCI Machine Learning Repository — Air Quality Dataset (De Vito et al., 2008). Hourly measurements from March 2004 to April 2005 at one urban site in Italy. The project-provided CSV was partially pre-cleaned: heavily missing variables were dropped, rows without valid dates were removed, and remaining gaps were median-imputed. As noted in the EDA below, residual −200 sentinel codes survive in `CO_GT` and `C6H6_GT`; these two variables are not used as predictors or the response in our analysis, so they do not affect any model results.

Key variable groups: - *Sensors (predictors):* `PT08_S1_CO`, `PT08_S2_NMHC`, `PT08_S3_NOx`, `PT08_S4_NO2`, `PT08_S5_O3` - *Weather covariates:* `T` (temperature °C), `RH` (relative humidity %), `AH` (absolute humidity g/m³) - *Reference measurements:* `CO_GT`, `C6H6_GT`, `NOx_GT`, `NO2_GT` (certified analyzer readings — used only as context and as the response variable) - *Response:* `NO2_GT` (µg/m³)

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

plt.rcParams.update({'figure.dpi': 120, 'font.size': 10})
np.random.seed(42)
```

1.3 Section 1: EDA and Preprocessing

[ ]:
```
df = pd.read_csv('air_quality_italy.csv')

print('Shape:', df.shape)
print('\nDtypes:')
print(df.dtypes)
print('\nNull counts:')
print(df.isnull().sum())

# Confirm no residual -200 codes
neg200 = (df.select_dtypes('number') == -200).any()
print('\nAny -200 codes remaining?')
print(neg200[neg200].index.tolist() if neg200.any() else 'None - clean.')

# Build proper datetime index
df['timestamp'] = pd.to_datetime(df['Date']) + pd.to_timedelta(df['Hour'],␣
 ↪unit='h')
df = df.sort_values('timestamp').reset_index(drop=True)

print('\nDate range:', df['timestamp'].min(), 'to', df['timestamp'].max())
print('Total hours:', len(df))
```

1.3.1 Figure 1 — NO time series and distribution

The left panel shows strong diurnal cycling (morning and evening traffic peaks) and a seasonal drift: concentrations rise in autumn/winter when boundary-layer mixing is suppressed and fall in spring/summer. The right panel shows that raw NO _GT is mildly right-skewed (skewness 0.73), while the log transform over-corrects to a left-skewed distribution (skewness −1.09). Given that the log transform does not substantially improve symmetry, we fit all supervised models on the raw concentration scale and note the scale choice as a limitation.

[ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.5))

# Time series - weekly rolling mean overlaid for trend
axes[0].plot(df['timestamp'], df['NO2_GT'], lw=0.4, alpha=0.5,␣
 ↪color='steelblue', label='Hourly')
roll = df.set_index('timestamp')['NO2_GT'].rolling('7D').mean()
axes[0].plot(roll.index, roll.values, lw=1.5, color='darkblue', label='7-day␣
 ↪rolling mean')
axes[0].set_xlabel('Date')
axes[0].set_ylabel('NO _GT (µg/m³)')
axes[0].set_title('NO  Reference Concentration Over Time')
axes[0].legend(fontsize=8)
axes[0].tick_params(axis='x', rotation=30)

# Distribution - raw vs log
axes[1].hist(df['NO2_GT'], bins=60, alpha=0.6, color='steelblue', label='Raw␣
 ↪NO _GT', density=True)
log_vals = np.log(df['NO2_GT'].replace(0, np.nan).dropna())
ax2 = axes[1].twinx()
ax2.hist(log_vals, bins=60, alpha=0.4, color='darkorange', label='log(NO _GT)',␣
 ↪density=True)
ax2.set_ylabel('Density (log scale)', color='darkorange')
axes[1].set_xlabel('NO _GT (µg/m³)')
axes[1].set_ylabel('Density (raw scale)', color='steelblue')
axes[1].set_title('Distribution of NO _GT (raw vs log)')
lines1, labels1 = axes[1].get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
axes[1].legend(lines1 + lines2, labels1 + labels2, fontsize=8)

plt.tight_layout()
plt.savefig('fig1_no2_eda.png', bbox_inches='tight')
plt.show()

print(f"NO2_GT skewness: {df['NO2_GT'].skew():.2f}")
print(f"log(NO2_GT) skewness: {log_vals.skew():.2f}")
```

1.3.2 Figure 2 — Correlation heatmap

We compute Pearson correlations across three variable blocks: the five PT08 sensors, the four reference-analyzer GT variables, and the three meteorological covariates. The block structure motivates both PCA (the sensor block is internally correlated, suggesting low intrinsic rank) and the supervised analysis (cross-block correlation between sensors and NO _GT supports sensor-based prediction).

[ ]:
```
SENSORS  = ['PT08_S1_CO', 'PT08_S2_NMHC', 'PT08_S3_NOx', 'PT08_S4_NO2',␣
 ↪'PT08_S5_O3']
GT_VARS  = ['CO_GT', 'C6H6_GT', 'NOx_GT', 'NO2_GT']
```

```
WEATHER  = ['T', 'RH', 'AH']
TARGET   = 'NO2_GT'

corr_cols = SENSORS + GT_VARS + WEATHER
corr_mat  = df[corr_cols].corr()

fig, ax = plt.subplots(figsize=(9, 7))
mask = np.triu(np.ones_like(corr_mat, dtype=bool), k=1)  # keep lower triangle␣
 ↪+ diagonal
sns.heatmap(
    corr_mat, mask=~(~mask),  # show full matrix
    annot=True, fmt='.2f', annot_kws={'size': 7},
    cmap='RdBu_r', center=0, vmin=-1, vmax=1,
    linewidths=0.3, ax=ax
)
ax.set_title('Figure 2: Correlation Heatmap - Sensors, Reference Analyzers, and␣
 ↪Weather', pad=12)

# Draw block separator lines
for pos in [len(SENSORS), len(SENSORS)+len(GT_VARS)]:
    ax.axhline(pos, color='black', lw=1.5)
    ax.axvline(pos, color='black', lw=1.5)

plt.tight_layout()
plt.savefig('fig2_corr_heatmap.png', bbox_inches='tight')
plt.show()
```

1.4 Section 2: Chronological Train / Test Split

Because the data are hourly time-series with strong autocorrelation and seasonal drift, a random split would leak information from future to past. We use a chronological holdout: the first 75% of sorted observations form the training set, the remaining 25% form the test set. The split point is stated below. All preprocessing statistics (scaling means/SDs, PCA directions, regularization hyperparameters) are estimated on training data only and applied to test data without re-fitting.

[ ]:
```
n = len(df)
split_idx = int(np.floor(0.75 * n))

train = df.iloc[:split_idx].copy()
test  = df.iloc[split_idx:].copy()

print(f'Total observations : {n}')
print(f'Training set       : {len(train)} rows  ({train["timestamp"].min().
 ↪date()} to {train["timestamp"].max().date()})')
print(f'Test set           : {len(test)}  rows  ({test["timestamp"].min().
 ↪date()} to {test["timestamp"].max().date()})')
```

```
print(f'Split point        : after index {split_idx-1}, split date␣
  ↪{test["timestamp"].min().date()}')
```

1.5 Section 3: PCA on the Sensor Block

We run PCA exclusively on the five PT08 sensor variables. Weather variables are excluded so that the structural story stays clean: the question here is *how many independent axes of variation exist in the sensor array itself?* Weather enters only as covariates in the supervised stage.

Standardization (zero mean, unit variance) uses training-set statistics only; the same transformation is applied to the test sensors.

[ ]:
```
# Fit scaler on train sensors only
scaler_pca = StandardScaler()
train_sensors_sc = scaler_pca.fit_transform(train[SENSORS])
test_sensors_sc  = scaler_pca.transform(test[SENSORS])

# Fit PCA on scaled train sensors
pca = PCA(n_components=5)
train_pcs = pca.fit_transform(train_sensors_sc)
test_pcs  = pca.transform(test_sensors_sc)

var_exp = pca.explained_variance_ratio_
cum_var = np.cumsum(var_exp)

print('Variance explained per PC:')
for i, (v, c) in enumerate(zip(var_exp, cum_var)):
    print(f'  PC{i+1}: {v:.3f}  (cumulative: {c:.3f})')

loadings = pd.DataFrame(
    pca.components_.T,
    index=SENSORS,
    columns=[f'PC{i+1}' for i in range(5)]
)
print('\nLoadings matrix:')
print(loadings.round(3))
```

1.5.1 Figure 3 — Scree plot and PC loadings

[ ]:
```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Scree plot
pcs = np.arange(1, 6)
axes[0].bar(pcs, var_exp, alpha=0.7, color='steelblue', label='Individual')
axes[0].plot(pcs, cum_var, 'o-', color='darkred', lw=2, label='Cumulative')
axes[0].axhline(0.9, color='gray', ls='--', lw=1, label='90% threshold')
axes[0].set_xticks(pcs)
axes[0].set_xticklabels([f'PC{i}' for i in pcs])
```

```
axes[0].set_ylabel('Proportion of variance explained')
axes[0].set_title('Scree Plot - Sensor PCA')
axes[0].legend(fontsize=8)
axes[0].set_ylim(0, 1.05)

# PC1 and PC2 loadings
x = np.arange(len(SENSORS))
width = 0.35
sensor_labels = [s.replace('PT08_S', 'S').replace('_CO','\n(CO)').
 ↪replace('_NMHC','\n(NMHC)')
                 .replace('_NOx','\n(NOx)').replace('_NO2','\n(NO2)').
 ↪replace('_O3','\n(O3)') for s in SENSORS]
axes[1].bar(x - width/2, loadings['PC1'], width, alpha=0.8, color='steelblue',␣
 ↪label='PC1')
axes[1].bar(x + width/2, loadings['PC2'], width, alpha=0.8, color='darkorange',␣
 ↪label='PC2')
axes[1].set_xticks(x)
axes[1].set_xticklabels(sensor_labels, fontsize=8)
axes[1].axhline(0, color='black', lw=0.8)
axes[1].set_ylabel('Loading')
axes[1].set_title('PC1 and PC2 Loadings')
axes[1].legend(fontsize=8)

plt.tight_layout()
plt.savefig('fig3_pca.png', bbox_inches='tight')
plt.show()
```

1.5.2 Interpretation of PCA structure

PC1 (84% of variance) loads positively and nearly equally on four of the five sensors (+0.42 to +0.47), but *negatively* on PT08_S3_NOx (−0.43). This sign flip is physically meaningful: PT08_S3 is a metal-oxide sensor tuned to NOx, which responds inversely to ambient NO oxidising conditions compared to the other sensors. PC1 therefore represents the overall combustion/traffic pollution load, with PT08_S3 providing a counteracting signal. The dominant share of variance captured by a single component confirms that the sensor array has very low intrinsic rank — it is primarily measuring one underlying process.

PC2 (7% of variance) is strongly dominated by PT08_S4_NO2 (+0.84), with PT08_S3_NOx (+0.39) and PT08_S5_O3 (−0.38) as secondary contributors. This axis separates the NO -specific sensor from the ozone sensor, reflecting the chemical antagonism between NO and O in urban photochemistry. Together PC1 and PC2 explain 91% of sensor variance; PC3–PC5 each capture less than 6%.

The low-rank structure motivates PC regression: projecting onto the first few PCs substitutes correlated raw sensors with orthogonal summary axes, which can stabilise regression coefficients. Whether this leads to better out-of-sample prediction depends on whether the discarded PCs carry useful signal — we test this directly in Section 5.

1.6 Section 4: Supervised Models

We fit four models predicting `NO2_GT` (raw scale), each using the five sensors plus T, RH, AH as predictors. All predictors are standardised using a separate scaler fit on training data only (independent of the PCA scaler above, since the supervised scaler covers all 8 variables).

- **OLS:** baseline, no regularisation
- **Ridge:** L2 penalty, selected by 5-fold CV on training set
- **Lasso:** L1 penalty, selected by 5-fold CV on training set
- **PC regression:** OLS on first k PCs + weather, k selected by 5-fold CV on training set

[ ]:
```
FEATURES = SENSORS + WEATHER  # 8 predictors

# Scaler for supervised models (all 8 features)
scaler_sup = StandardScaler()
X_train = scaler_sup.fit_transform(train[FEATURES])
X_test  = scaler_sup.transform(test[FEATURES])
y_train = train[TARGET].values
y_test  = test[TARGET].values

print(f'X_train shape: {X_train.shape}')
print(f'X_test  shape: {X_test.shape}')
```

[ ]:
```
# --- OLS ---
ols = LinearRegression()
ols.fit(X_train, y_train)
ols_train_pred = ols.predict(X_train)
ols_test_pred  = ols.predict(X_test)

# --- Ridge (CV on train) ---
alphas = np.logspace(-3, 4, 80)
ridge = RidgeCV(alphas=alphas, cv=5, scoring='neg_root_mean_squared_error')
ridge.fit(X_train, y_train)
ridge_train_pred = ridge.predict(X_train)
ridge_test_pred  = ridge.predict(X_test)
print(f'Ridge best  : {ridge.alpha_:.4f}')

# --- Lasso (CV on train) ---
lasso = LassoCV(cv=5, max_iter=20000, n_alphas=100, random_state=42)
lasso.fit(X_train, y_train)
lasso_train_pred = lasso.predict(X_train)
lasso_test_pred  = lasso.predict(X_test)
print(f'Lasso best  : {lasso.alpha_:.6f}')

# Lasso retained vs zeroed coefficients
lasso_coefs = pd.Series(lasso.coef_, index=FEATURES)
print('\nLasso coefficients:')
print(lasso_coefs.round(4))
```

```
zeroed = lasso_coefs[lasso_coefs == 0].index.tolist()
retained = lasso_coefs[lasso_coefs != 0].index.tolist()
print(f'\nZeroed out: {zeroed}')
print(f'Retained  : {retained}')
```

[ ]:
```
# --- PC Regression: choose k by 5-fold CV on train ---
# PC scores from the PCA scaler (sensor block only); append scaled weather
weather_scaler = StandardScaler()
train_weather_sc = weather_scaler.fit_transform(train[WEATHER])
test_weather_sc  = weather_scaler.transform(test[WEATHER])

kf = KFold(n_splits=5, shuffle=False)  # no shuffle - preserve time order␣
 ↪within train
cv_rmse = {}
for k in range(1, 6):
    X_pc_train = np.hstack([train_pcs[:, :k], train_weather_sc])
    scores = cross_val_score(
        LinearRegression(), X_pc_train, y_train,
        cv=kf, scoring='neg_root_mean_squared_error'
    )
    cv_rmse[k] = -scores.mean()
    print(f'k={k}  CV RMSE={cv_rmse[k]:.3f}')

best_k = min(cv_rmse, key=cv_rmse.get)
print(f'\nBest k = {best_k}  (CV RMSE = {cv_rmse[best_k]:.3f})')

X_pc_train_best = np.hstack([train_pcs[:, :best_k], train_weather_sc])
X_pc_test_best  = np.hstack([test_pcs[:, :best_k],  test_weather_sc])
pcr = LinearRegression()
pcr.fit(X_pc_train_best, y_train)
pcr_train_pred = pcr.predict(X_pc_train_best)
pcr_test_pred  = pcr.predict(X_pc_test_best)
```

1.7 Section 5: Comparison

1.7.1 Table 1 — Model performance on chronological test set

[ ]:
```
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

results = pd.DataFrame({
    'Model': ['OLS (baseline)', f'Ridge ( ={ridge.alpha_:.3f})', f'Lasso␣
 ↪( ={lasso.alpha_:.4f})', f'PC Regression (k={best_k})'],
    'Train RMSE': [
        rmse(y_train, ols_train_pred),
        rmse(y_train, ridge_train_pred),
        rmse(y_train, lasso_train_pred),
```

```
        rmse(y_train, pcr_train_pred),
    ],
    'Test RMSE': [
        rmse(y_test, ols_test_pred),
        rmse(y_test, ridge_test_pred),
        rmse(y_test, lasso_test_pred),
        rmse(y_test, pcr_test_pred),
    ],
    'Test R²': [
        r2_score(y_test, ols_test_pred),
        r2_score(y_test, ridge_test_pred),
        r2_score(y_test, lasso_test_pred),
        r2_score(y_test, pcr_test_pred),
    ]
})
results[['Train RMSE', 'Test RMSE', 'Test R²']] = results[['Train RMSE', 'Test␣
 ↪RMSE', 'Test R²']].round(3)
print('Table 1: Model Comparison')
print(results.to_string(index=False))
```

1.7.2 Figure 4 — Predicted vs actual on the test set

[ ]:
```
model_preds = [
    ('OLS (baseline)',          ols_test_pred,   'steelblue'),
    (f'Ridge ( ={ridge.alpha_:.3f})', ridge_test_pred, 'forestgreen'),
    (f'Lasso ( ={lasso.alpha_:.4f})', lasso_test_pred, 'darkorange'),
    (f'PC Regression (k={best_k})',   pcr_test_pred,   'crimson'),
]

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

lo = min(y_test.min(), min(p.min() for _, p, _ in model_preds)) - 5
hi = max(y_test.max(), max(p.max() for _, p, _ in model_preds)) + 5

for ax, (name, preds, color) in zip(axes, model_preds):
    ax.scatter(y_test, preds, alpha=0.15, s=4, color=color)
    ax.plot([lo, hi], [lo, hi], 'k--', lw=1, label='Perfect')
    r2 = r2_score(y_test, preds)
    rmse_val = rmse(y_test, preds)
    ax.set_title(f'{name}\nRMSE={rmse_val:.2f}, R²={r2:.3f}', fontsize=9)
    ax.set_xlabel('Actual NO _GT')
    ax.set_ylabel('Predicted NO _GT')
    ax.set_xlim(lo, hi)
    ax.set_ylim(lo, hi)
    ax.set_aspect('equal', 'box')
```

```
fig.suptitle('Figure 4: Predicted vs Actual NO  on Chronological Test Set',␣
  ↪fontsize=11, y=1.01)
plt.tight_layout()
plt.savefig('fig4_pred_vs_actual.png', bbox_inches='tight')
plt.show()
```

1.7.3 Discussion: What each method reveals

All four models achieve nearly identical test performance (RMSE 40.4 µg/m³, R² 0.35), and all show a large train-to-test RMSE gap (~25 train vs. ~40 test). The gap is larger than typical overfitting: the training set covers spring through autumn 2004, while the test set is winter 2004–2005. Seasonal changes in boundary-layer dynamics, photochemistry, and sensor behaviour mean the test distribution is systematically different from training — this is the sensor drift and distributional shift noted in the limitations.

OLS (baseline): Even without regularisation, OLS is not badly overfit relative to the regularised alternatives — suggesting the eight predictors are informative but the main challenge is distributional shift, not in-sample variance.

Ridge (49.7): The large optimal indicates the CV on training data favours heavy shrinkage, consistent with high inter-sensor collinearity. Ridge achieves the marginally best test RMSE (40.39), but the improvement over OLS is negligible (~0.1 µg/m³), confirming that coefficient instability from multicollinearity is not the binding constraint.

Lasso (0.025): Lasso retains all eight predictors — no sensor is zeroed out. This is noteworthy in light of PCA: even though PC1 explains 84% of sensor variance, Lasso assigns nonzero weight to all five sensors individually. The reason is that Lasso optimises for predictive relevance, not for capturing sensor variance. Every sensor contributes marginal information about NO _GT beyond what the others provide, so Lasso does not drop any. Compare this directly with the PCA loadings: PT08_S3_NOx loads negatively on PC1 but carries a nonzero Lasso coefficient (−3.90), confirming it adds predictive signal that the variance-maximising PC1 direction partially suppresses.

PC Regression (k = 5, chosen by CV): The cross-validation RMSE decreases monotonically from k = 1 (29.2) to k = 5 (26.7), meaning every PC — including PC3–PC5, which together explain only 9% of sensor variance — contributes useful information about NO _GT. With k = 5, PC regression is algebraically equivalent to OLS on the scaled raw sensors (it spans the same column space), explaining why their test metrics are identical. This is the core tension between PCA and Lasso: PCA chose to discard the low-variance PCs on structural grounds, but CV reveals those directions are predictively relevant.

Primary metric: test RMSE. R^2 reported for interpretability. The near-identical performance across methods, combined with the large train-test gap, suggests that the binding problem is seasonal distributional shift rather than regularisation choice.

1.8 Section 6: Limitations

1. **Residual autocorrelation.** Hourly observations are strongly serially correlated. OLS standard errors assume independence, which they do not have. Newey–West (HAC) standard errors would correct inference without changing point estimates; this was not implemented. The CV folds also do not account for serial correlation, so reported CV RMSEs are optimistic.

2. **Seasonal distributional shift / sensor drift.** The large train-test RMSE gap (~25 vs. ~40) is not explained by overfitting alone — it reflects systematic differences between the training period (spring–autumn 2004) and the test period (winter 2004–2005). Electrochemical sensors are known to drift over months of operation and respond differently under winter atmospheric chemistry. This limits confidence in the absolute test RMSE numbers.

3. **Median imputation compresses tails.** The project-provided CSV replaced missing values with column medians. Imputing at the median artificially pulls extreme observations toward the center, understating tail variance and potentially improving in-sample fit while masking true data sparsity at high-pollution episodes.

4. **Single monitoring site.** All data come from one urban location in Italy over one year. Results may not generalise to other cities, sensor deployments, climates, or traffic profiles. Cross-site validation would be required to assess external validity.

5. **Reference measurement error.** The GT variables are certified analyzer readings, but they carry calibration uncertainty and sampling noise. Treating `NO2_GT` as noiseless ground truth introduces errors-in-variables bias — the true predictive signal from the sensors may be somewhat higher than our R^2 values suggest.

6. **Log transform omitted.** The raw NO _GT distribution is mildly right-skewed (skewness 0.73), but the log transform over-corrects to a left-skewed distribution (skewness -1.09). We therefore modelled on the raw scale. A Box–Cox transformation could achieve better symmetry, but was not explored.